



Atelier Développement Mobile

Kotlin — Android

PokéDex App

Durée estimée

8 à 12 heures de travail

Technologie principale

Kotlin / Android Studio

API utilisée

PokéAPI — <https://pokeapi.co>

Rendu

Dépôt Git + APK de démonstration

1. Contexte et Objectifs du Projet

1.1 Mise en situation

Dans cet atelier, vous allez concevoir et développer une application Android native en Kotlin : un Pokédex interactif. L'application consommera les données de l'API publique PokéAPI, affichera une liste de Pokémon avec leurs détails, et permettra à l'utilisateur de gérer une liste de favoris persistée localement.

Ce projet couvre les compétences fondamentales du développement Android moderne :

- Architecture logicielle (MVVM, Clean Architecture)
- Programmation asynchrone avec Kotlin Coroutines et Flow
- Consommation d'API REST avec Retrofit
- Persistance de données locale avec Room ou DataStore
- Navigation entre écrans avec Jetpack Navigation
- Construction d'interfaces avec Jetpack Compose ou XML/ViewBinding

1.2 L'API PokéAPI

Documentation officielle : <https://pokeapi.co/>

Endpoint liste des Pokémon : GET <https://pokeapi.co/api/v2/pokemon?limit=100&offset=0>

Endpoint détail d'un Pokémon : GET <https://pokeapi.co/api/v2/pokemon/{id}>

L'API est gratuite, ne nécessite pas d'authentification et dispose d'une pagination.

L'API retourne notamment pour chaque Pokémon :

- Son identifiant et son nom
- Ses types (feu, eau, plante, etc.)
- Ses statistiques de base (PV, attaque, défense, vitesse...)
- Ses sprites (images front/back, officiel artwork)
- Sa taille, son poids, ses capacités

2. Architecture & Bonnes Pratiques Exigées

Cette section est OBLIGATOIRE. La qualité architecturale sera évaluée au même titre que les fonctionnalités. Tout projet ne respectant pas ces contraintes se verra pénalisé.

2.1 Pattern MVVM obligatoire

Votre projet doit impérativement suivre le pattern MVVM (Model - View - ViewModel) :

Couche	Rôle	Exemples dans le projet
Model	Données, logique métier, accès réseau et base de données	Pokemon.kt, PokemonRepository, PokemonDao
ViewModel	Expose les états UI, orchestre les use cases, survit aux rotations d'écran	PokemonListViewModel, FavoritesViewModel
View	Affichage uniquement, observe le ViewModel via StateFlow ou LiveData	PokemonListFragment/Screen, DetailScreen

2.2 Structure des packages

Organisez votre projet en packages selon les responsabilités (feature-based ou layer-based) :

```
com.votreapp.pokedex/  
├─ data/  
│   ├── remote/      ← API Retrofit, DTOs  
│   ├── local/       ← Room Database, DAOs, Entities  
│   └─ repository/   ← Implémentation des repositories  
├─ domain/  
│   ├── model/       ← Modèles métier (Pokemon, PokemonDetail...)  
│   ├── repository/   ← Interfaces des repositories  
│   └─ usecase/      ← Use cases (GetPokemonListUseCase...)  
├─ presentation/  
│   └─ list/         ← PokemonListViewModel + UI
```

```
|   └─ detail/      ← PokemonDetailViewModel + UI
|   └─ favorites/   ← FavoritesViewModel + UI
└─ di/              ← Injection de dépendances (Hilt)
```

2.3 Interfaces et injection de dépendances

Chaque repository doit être défini par une interface dans la couche domain, et implémenté dans la couche data. L'injection de dépendances est obligatoire via Hilt :

- ✓ PokemonRepository est une interface dans domain/repository/
- ✓ PokemonRepositoryImpl implémente cette interface dans data/repository/
- ✓ Les ViewModels reçoivent leurs dépendances via le constructeur (pas de new/getInstance)
- ✓ Hilt fournit les dépendances avec @HiltViewModel et @Inject

2.4 Programmation réactive et asynchrone

- Utilisez exclusivement des Kotlin Coroutines pour les opérations asynchrones (appels réseau, accès BDD)
- Utilisez StateFlow ou Flow pour exposer les états depuis le ViewModel
- N'effectuez jamais d'appel réseau sur le thread principal
- Gérez proprement les états de chargement : Loading, Success, Error

2.5 Gestion des erreurs

Chaque appel réseau doit être encapsulé dans un wrapper d'état :

```
sealed class UiState<out T> {
    object Loading : UiState<Nothing>()
    data class Success<T>(val data: T) : UiState<T>()
    data class Error(val message: String) : UiState<Nothing>()
}
```

L'interface utilisateur doit réagir à chacun de ces états : afficher un loader, les données ou un message d'erreur.

3. Spécifications Fonctionnelles

3.1 Navigation principale

L'application comporte une navigation par onglets en bas de l'écran (Bottom Navigation Bar) avec deux onglets :



Chacun de ces onglets possède son propre back stack de navigation, géré via Jetpack Navigation Component.

3.2 Écran 1 — Liste du Pokédex

3.2.1 Affichage de la liste

Cet écran est le point d'entrée de l'onglet Pokédex. Il affiche la liste des Pokémon récupérés depuis PokéAPI.

Chaque item de la liste doit afficher :

- Le numéro du Pokémon dans le Pokédex (ex : #025)
- Le sprite/image du Pokémon (chargée de manière asynchrone)
- Le nom du Pokémon (formaté avec majuscule initiale)
- Ses types sous forme de texte ou badge simple (ex : Feu, Vol)

Suggestion UI : une RecyclerView avec LinearLayoutManager (liste verticale) est suffisante. Chargez un premier lot de 100 Pokémon au démarrage via le paramètre ?limit=100 de l'API. La pagination n'est pas exigée — concentrez-vous sur la liste et les filtres.

3.2.2 Système de filtre

Un champ de recherche doit être présent en haut de l'écran pour filtrer la liste :

- Filtrage par nom (recherche insensible à la casse)
- Filtrage par type via un menu déroulant ou des chips sélectionnables
- Les filtres s'appliquent sur la liste déjà chargée, sans nouvel appel réseau

3.3 Écran 2 — Fiche Détail d'un Pokémon

Cet écran est accessible en cliquant sur un Pokémon dans la liste.

Il doit afficher :

- Le sprite ou l'image officielle du Pokémon
- Son nom, son numéro Pokédex, ses types
- Sa taille et son poids
- Ses statistiques de base : PV, Attaque, Défense, Att. Spé, Déf. Spé, Vitesse (valeurs numériques, pas d'animation obligatoire)

- Un bouton pour ajouter / retirer des favoris, avec un état visuel distinct (ex : icône pleine vs vide)

3.4 Écran 3 — Liste des Favoris

Accessible depuis le deuxième onglet, cet écran affiche uniquement les Pokémons marqués comme favoris.

3.4.1 Fonctionnement

- La liste des favoris réutilise le même design que la liste du Pokédex
- Elle est alimentée depuis la base de données locale (Room)
- Un Pokémon ajouté ou retiré depuis la fiche détail se reflète automatiquement ici via Flow
- Si aucun favori : afficher un message indiquant que la liste est vide
- Un clic sur un favori ouvre sa fiche détail

3.4.2 Persistance locale

La persistance des favoris doit être réalisée avec Room Database :

Créez une Entity FavoritePokemon dans la couche data/local/
Créez un FavoritePokemonDao avec les méthodes insert, delete, getAll, isFavorite
Exposez un Flow<List<FavoritePokemon>> depuis le DAO pour réactivité automatique
Le ViewModel consomme ce Flow et l'expose en tant que StateFlow<UiState<List<Pokemon>>>

4. Stack Technique Recommandée

4.1 Dépendances Gradle suggérées

Voici les bibliothèques recommandées pour ce projet. Leur utilisation n'est pas toutes obligatoires, mais certaines sont fortement conseillées :

Catégorie	Librairie	Usage
Réseau	Retrofit + Gson/Moshi	Consommation de l'API REST
Réseau	OkHttp + Logging Interceptor	Débogage des requêtes HTTP
Images	Coil ou Glide	Chargement asynchrone des sprites
DI	Hilt (Dagger Hilt)	Injection de dépendances
BDD locale	Room	Persistance des favoris
Async	Kotlin Coroutines + Flow	Programmation réactive

Navigation	Jetpack Navigation Component	Navigation entre fragments/composables
ViewModel	Lifecycle ViewModel + StateFlow	Architecture MVVM
(Bonus)	Paging 3	Pagination avancée de la liste

4.2 Choix de l'UI

Vous avez le choix entre deux approches pour l'interface utilisateur :

Option A — XML + ViewBinding	Option B — Jetpack Compose ★
Approche classique, plus de ressources en ligne, idéale si vous débutez	Approche moderne recommandée par Google. Bonus de points si maîtrisée.

5. Contraintes et Règles du Projet

5.1 Contraintes obligatoires

- Le projet doit compiler et fonctionner sur un émulateur Android API 26+ ou un appareil réel
- Aucune API key n'est requise pour PokéAPI — ne pas en créer une inutilement
- Le code doit être versionné sur un dépôt Git (GitHub, GitLab, Bitbucket...)
- Le dépôt doit contenir un fichier README.md décrivant le projet, les librairies utilisées et les choix techniques
- Pas de code mort ou commenté sans justification
- Les fichiers de build (build.gradle) doivent utiliser les versions stables des dépendances

5.2 Interdictions

- Il est interdit d'utiliser des frameworks cross-platform (Flutter, React Native, Xamarin)
- Il est interdit d'instancier manuellement les ViewModels (utiliser Hilt)
- Il est interdit de faire des appels réseau sur le Main Thread
- Il est interdit de stocker les favoris dans SharedPreferences sans migration vers Room

5.3 Travail individuel ou en binôme

Le projet peut être réalisé seul ou en binôme. En binôme, le README devra indiquer clairement la répartition des tâches, et les deux membres devront être en mesure d'expliquer l'ensemble du code lors de la soutenance.

6. Modalités de Rendu

6.1 Ce qui doit être rendu

1. Le lien vers le dépôt Git public (ou accès partagé avec l'enseignant)
2. Un APK de démonstration fonctionnel (mode debug accepté)
3. Un fichier README.md complet

6.2 Contenu du README

- Description du projet et des fonctionnalités implémentées
- Stack technique utilisée (bibliothèques et versions)
- Choix architecturaux justifiés
- Difficultés rencontrées et solutions apportées
- Captures d'écran de l'application
- Répartition des tâches (si binôme)

7. Conseils et Ressources

7.1 Conseils pratiques

Commencez par mettre en place l'architecture avant de coder les fonctionnalités.
Testez chaque couche séparément : d'abord le Repository, puis le ViewModel, puis l'UI.
Utilisez les logs et le Logcat d'Android Studio pour déboguer vos appels réseau.
Commitez régulièrement sur Git avec des messages de commit clairs et descriptifs.
Ne cherchez pas à tout faire parfaitement du premier coup : itérez.

7.2 Ressources recommandées

- Documentation Android Developers : <https://developer.android.com>
- Guide d'architecture Android : <https://developer.android.com/topic/architecture>
- Documentation PokéAPI : <https://pokeapi.co/docs/v2>
- Documentation Retrofit : <https://square.github.io/retrofit/>
- Documentation Hilt : <https://dagger.dev/hilt/>
- Documentation Room : <https://developer.android.com/training/data-storage/room>
- Documentation Kotlin Coroutines : <https://kotlinlang.org/docs/coroutines-overview.html>

- Codelabs Android officiels : <https://developer.android.com/codelabs>

Bon courage et bonne chance ! 🚀

N'hésitez pas à poser vos questions pendant les séances d'encadrement.